

Modelling and Simulation

Lecture 5: Lattice Boltzmann modeling of fluid flow

1. Computational Fluid Dynamics: Overview

What's a fluid?

A fluid ...

- ... is a substance that continually deforms under stresses.
- ... resists deformations only lightly, because of viscosity.
- ... can adopt the shape of any container into which it flows.

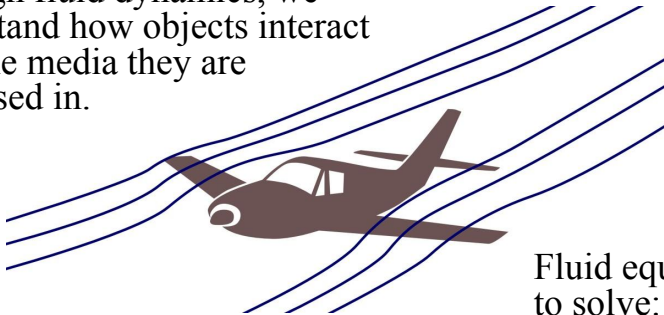
Fluids include:

- Liquids
- Gases
- Plasmas

Fluid = Continuum Model

Why fluid dynamics?

Through fluid dynamics, we understand how objects interact with the media they are immersed in.



Fluid equations are difficult to solve: this is a case for numerical simulation.

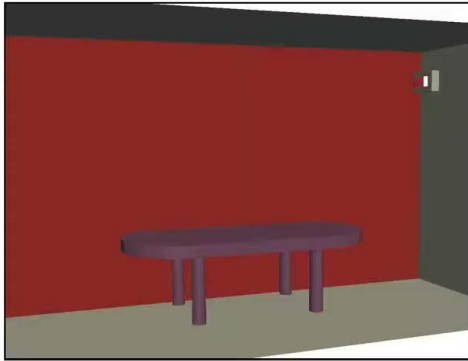
Fluid dynamics is everywhere

Important application areas are found

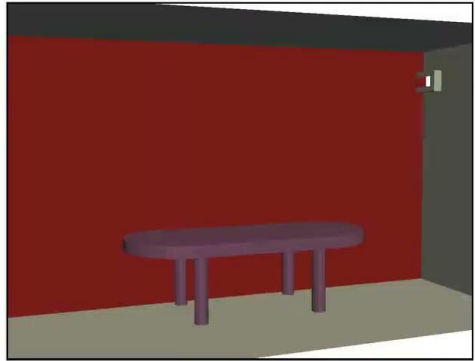
- In physics.
- In biology and medical physics.
- In chemistry.
- In geology.
- ... and many more.

Example 1: Heat convection

Air-conditioner in a meeting room

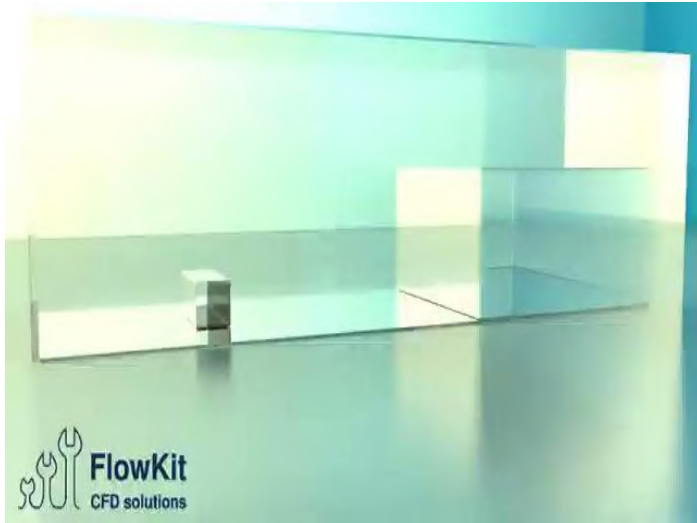


Fixed air-conditioner

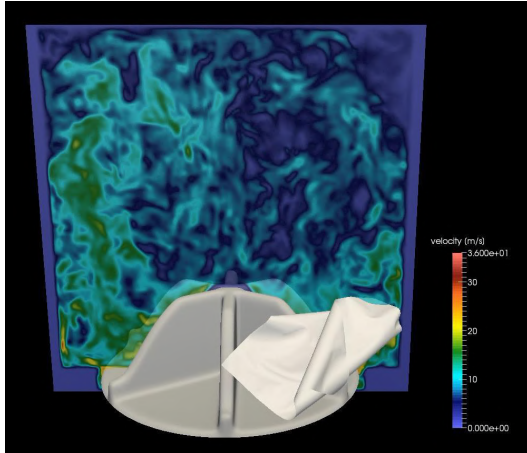


Sweeping air-conditioner

Example 2: Collapse of a water column



Example 3: Washing machine



Washing machine:
velocity field.

Simulation:

- water fluid flow
- rotating agitator
- flexible tissue

2. Computational Fluid Dynamics: Equations and challenges

The Navier-Stokes equations

The basis: Navier-Stokes equations for incompressible flow.

They don't account for

- Fluid compressibility in gases.
- Thermal effects.
- Chemical reactions.
- ... many more.

And yet, they are very frequently and successfully used, for both liquids and gases.

The Navier-Stokes equations

$$\partial_t \mathbf{u} - (\mathbf{u} \cdot \nabla) \mathbf{u} = -\frac{1}{\rho_0} \nabla p + \nu \Delta \mathbf{u}$$

$\mathbf{u}(\mathbf{x}, t)$	Velocity
$p(\mathbf{x}, t)$	Pressure
ν	Viscosity

$$\nabla \cdot \mathbf{u} = 0$$

Continuity equation:
Fluid incompressibility

The Navier-Stokes equations

$$\partial_t \mathbf{u} - (\mathbf{u} \cdot \nabla) \mathbf{u} = -\frac{1}{\rho_0} \nabla p + \nu \Delta \mathbf{u}$$

Flow acceleration

Convective acceleration

Pressure gradient

Viscous dissipation

$$\nabla \cdot \mathbf{u} = 0$$

Continuity equation:
Fluid incompressibility

Dimensionless formulation

Choose a...

Characteristic
velocity: U

Characteristic
length: L

$$\partial_t \mathbf{u} + (\mathbf{u} \cdot \nabla) \mathbf{u} = -\frac{1}{\rho_0} \nabla p + \nu \Delta \mathbf{u}$$

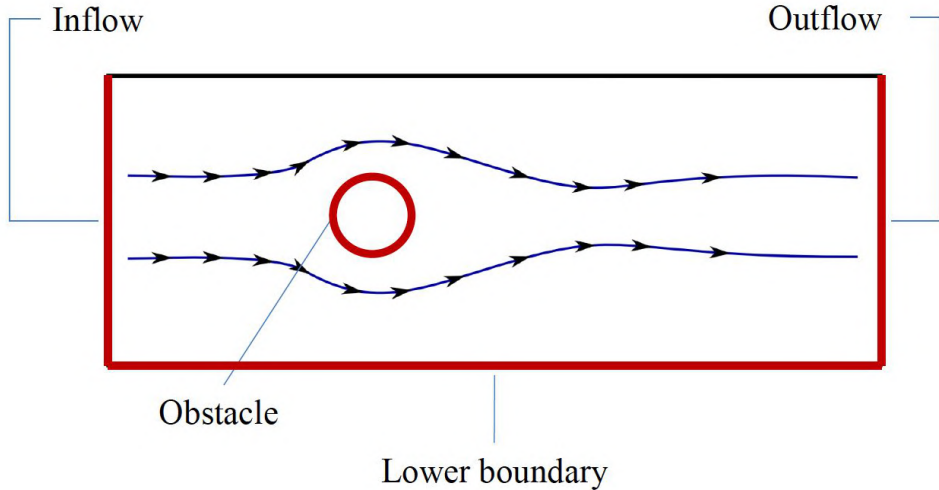

$$\begin{aligned} \vec{u}^* &= \frac{\vec{u}}{U} & \partial_t^* &= \frac{L}{U} \partial_t \\ p^* &= \frac{p}{\rho_0 U^2} & \vec{\nabla}^* &= L \vec{\nabla} \end{aligned}$$

Reynolds number:

$$Re = \frac{UL}{\nu}$$

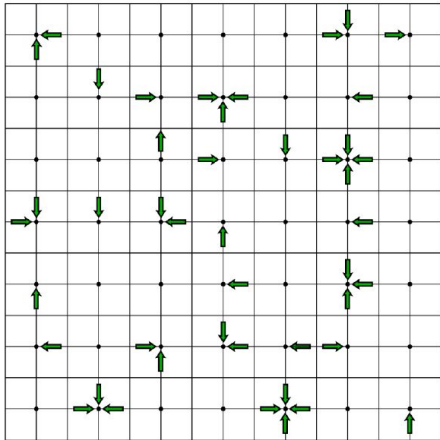

$$\partial_t^* \mathbf{u}^* + (\mathbf{u}^* \cdot \nabla^*) \mathbf{u}^* = -\nabla p^* + \frac{\nu}{UL} \Delta \mathbf{u}^*$$

Boundary Conditions



3. From Lattice Gas to Lattice Boltzmann

Lattice Gas Automata: Reminder



The lattice gas automaton HPP

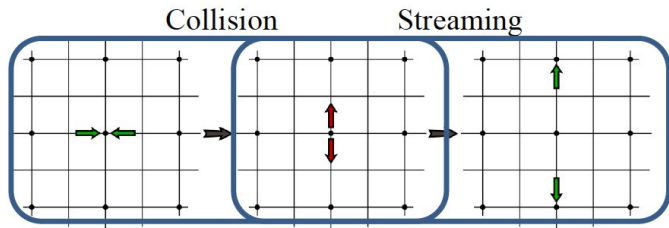
- Discrete Boolean states.
- Each cell is populated by four particles at most.
- Each of the four storage locations corresponds to a direction of propagation of the particle.

Comment

- This mesoscopic model describes the dynamics of a gas, not a liquid.

Lattice Gas Automata: Reminder

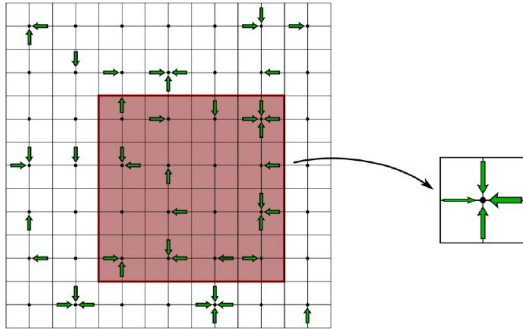
- When several particles meet on a node, they collide: all physics takes place here.
- After collision, particles are streamed to a nearest neighbor.



Our color scheme:

- ➡ Green: pre-collision
- ➡ Red: post-collision

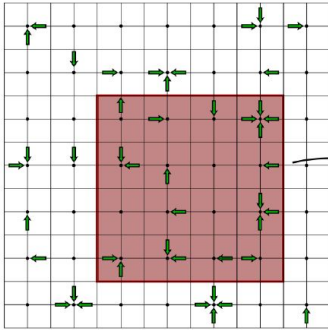
From discrete to continuum variables



In a lattice gas automaton:

- Run the simulation with discrete variables.
- At the end, extract macroscopic variables by taking averages.

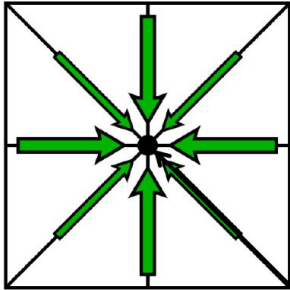
Lattice Boltzmann Method: Idea



- Run simulation with continuum variables right away, to save space.
- Continuum model is derived from discrete method by means of statistics.

- Particle density: real-valued.
- Arrow thickness stands for particle density.

Lattice Boltzmann Method: details

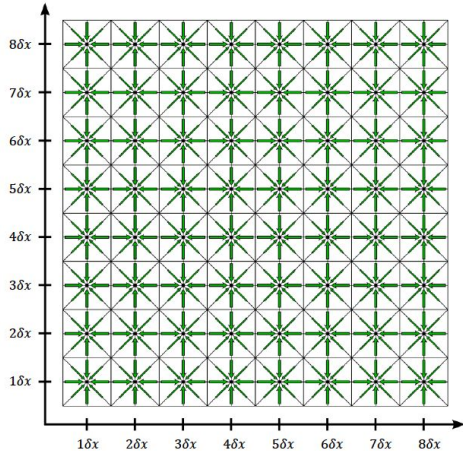


Lattice Boltzmann 2D model D2Q9:

- Four directions aren't quite enough.
- Nine directions: eight connections to nearest neighbors + «rest population».
- Variables representing particle densities are called «populations».

«Rest population» particles stay on a cell, don't travel to neighbors during streaming.

Lattice Boltzmann Variables



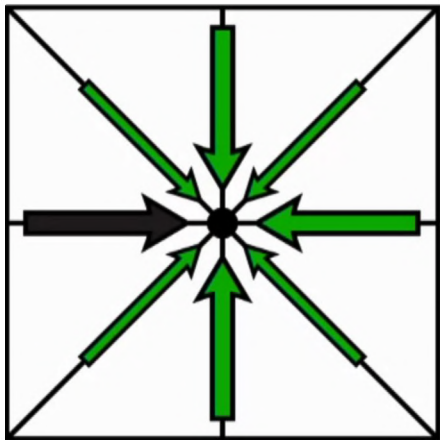
Space Discretization:

- Every cell represents flow variables at a point in space.
- Equal distance δx between cells in x- and y-direction.
- To represent space with 8×8 points, we need a total of $8 \times 8 \times 9 = 576$ floating point variables.

Python code to allocate memory:

```
f = zeros(9, 8, 8)
```

Collision step



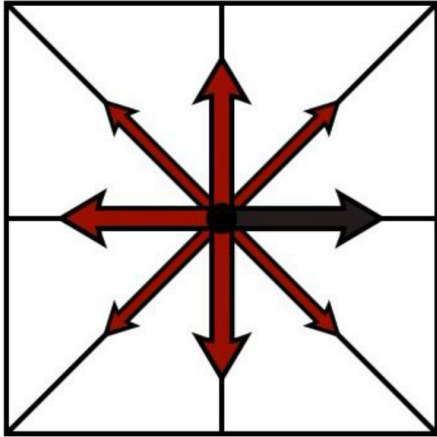
- Like in lattice gas: collision maps pre-collision populations to post-collision populations.
- All populations are modified during collision. Changes are not visible on video, because they are small perturbations.

Our color scheme:

 Green: pre-collision

 Red: post-collision

Collision step



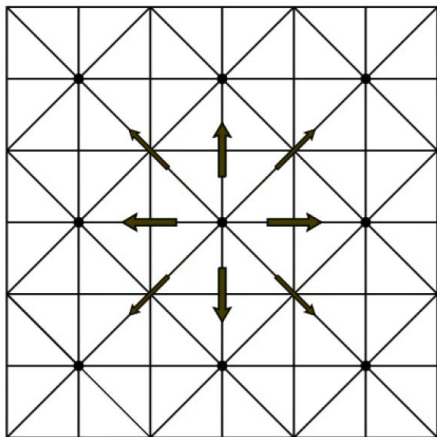
Model:

- Collision is instantaneous.
- Collision is localized at cell coordinates.

Our color scheme:

-  Green: pre-collision
-  Red: post-collision

Streaming step



Model:

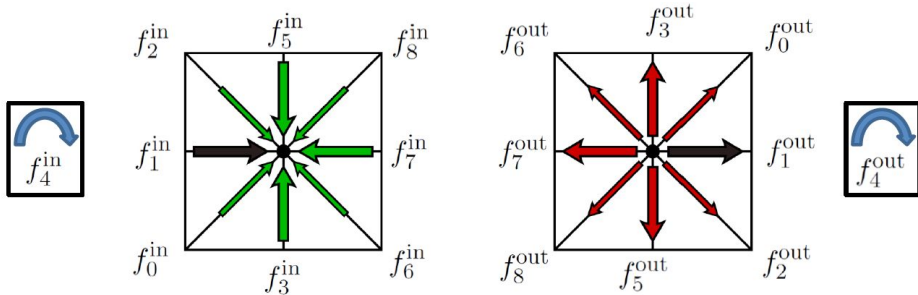
- Streaming takes the system from time t to time $t + \delta t$.
- Streaming includes nearest-neighbor access.

Our color scheme:

-  Green: pre-collision
-  Red: post-collision

4. Macroscopic Variables

Back to the collision step



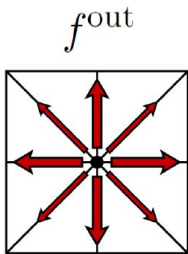
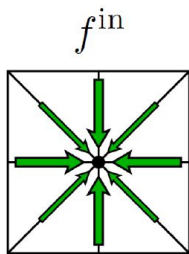
Pre-collision: $f_i^{\text{in}}(\mathbf{x}, t)$
«Incoming populations»

Post-collision: $f_i^{\text{out}}(\mathbf{x}, t)$
«Outgoing populations»

The index i runs from 0 to 8.

Pre- and post-collision: variables

In our Python code, we save incoming and outgoing populations in separate matrices.



Python code:

```
# assign some size  
nx, ny = ...  
fin = zeros(9, nx, ny)  
fout = zeros(9, nx, ny)
```

Density

Particle density is defined the same as in a lattice gas automaton:

$$\rho(\boldsymbol{x}, t) = \sum_{i=0}^8 f_i^{\text{in}}(\boldsymbol{x}, t)$$

Each cell has its own density, which is the sum of the nine populations.

Density

$$\rho(\boldsymbol{x}, t) = \sum_{i=0}^8 f_i^{\text{in}}(\boldsymbol{x}, t)$$

Python code:

```
rho = zeros((nx, ny))  
for ix in range(nx):  
    for iy in range(ny):  
        rho[ix, iy] = 0  
        for i in range(9):  
            rho[ix, iy] += fin[i, ix, iy]
```

**Simpler and faster, with NumPy
array-based syntax:**

```
rho = sum(fin, axis=0)
```

Pressure

Pressure is proportional to density, according to the ideal gas law, at constant temperature:

$$p = c_s^2 \rho$$

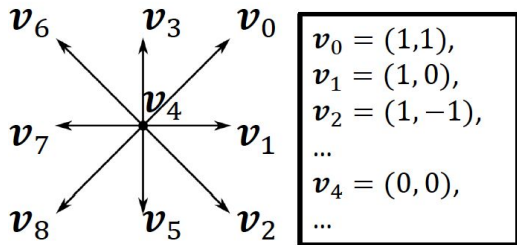
The speed of sound c_s is, in our D2Q9 model, a lattice constant:

$$c_s^2 = \frac{1}{3} \frac{\delta x^2}{\delta t^2}$$

Units: Say that δx has units of meters $[m]$ and δt has units of seconds $[s]$. Then pressure has units of $\frac{m^2}{s^2}$. Multiply this by the physical density of our fluid (e. g. $1 \frac{kg}{m^3}$) to obtain units of Pascal $\left(\frac{kg}{ms^2}\right)$.

Velocity

Let's introduce a set of 9 vectors:



They are called lattice velocities: If there's a cell at position $\mathbf{x}$, there's also one at $\mathbf{x} + \mathbf{v}_i \delta t$.

Again, velocity is defined just the same as in a lattice gas automaton:

$$u(\mathbf{x}, t) = \frac{1}{\rho(\mathbf{x}, t)} \frac{\delta x}{\delta t} \sum_{i=0}^8 \mathbf{v}_i f_i^{\text{in}}(\mathbf{x}, t)$$

Velocity

```
v = array([ [ 1, 1], [ 1, 0], [ 1, -1], [ 0, 1], [ 0, 0], [ 0, -1], [-1, 1], [-1, 0], [-1, -1] ])
```

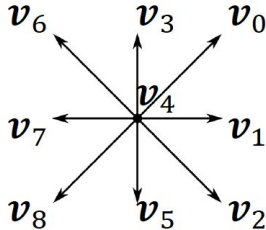
Traditional Python code:

```
u = zeros((2, nx, ny))
for ix in range(nx):
    for iy in range(ny):
        u[0, ix, iy] = 0
        u[1, ix, iy] = 0
        for i in range(9):
            u[0, ix, iy] += v[i, 0] * fin[i, ix, iy]
            u[1, ix, iy] += v[i, 1] * fin[i, ix, iy]
        u[0, ix, iy] /= rho[ix, iy]
        u[1, ix, iy] /= rho[ix, iy]
```

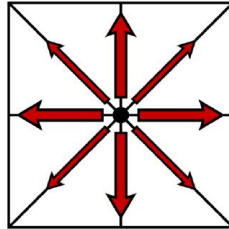
Simpler and faster, with NumPy array-based syntax:

```
u = zeros((2, nx, ny))
for i in range(9):
    u[0, :, :] += v[i, 0] * fin[i, :, :]
    u[1, :, :] += v[i, 1] * fin[i, :, :]
u /= rho
```


Don't be confused



- These are 9 2D vectors.
- They have constant values.

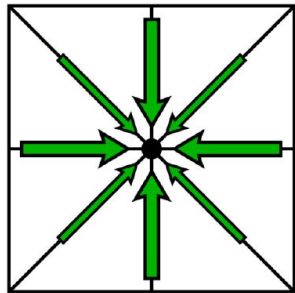


- These are 9 scalar values.
- Their value is time-dependent.

6. Collision step: the BGK model

What's the value of the populations?

- So far, we have interpreted populations as particle averages of a lattice gas automaton.
- Alternative interpretation: populations are probability densities of particles in a real gas.
- In *kinetic theory*, gas densities are described by a single *probability density function* $f(\mathbf{x}, \mathbf{v}, t)$.
- The motion of f is described by the *Boltzmann equation*.

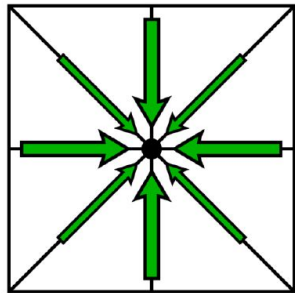


What's the value of the populations?

- When a gas is macroscopically at rest, or has a constant velocity $\mathbf{u}$, its molecules are still moving.
- The distribution of their velocities $\mathbf{v}$ at every point $\mathbf{x}$ in space is given by a *Maxwell-Boltzmann distribution*:

$$f(\mathbf{x}, \mathbf{v}, t) \sim e^{-|\mathbf{v}-\mathbf{u}|^2}.$$

- Even when it moves, a gas remains close to its equilibrium state. We consider its motion as a *perturbation around the equilibrium*.



Collision: BGK model

Inspired by kinetic theory, we formulate the collision term in lattice Boltzmann as a relaxation to local equilibrium:

$$f_i^{\text{out}} - f_i^{\text{in}} = -\omega \left(f_i^{\text{in}} - E(i, \rho, \vec{u}) \right)$$

- E is the local equilibrium. It depends on the macroscopic variables and has a different value for every direction i .
- The parameter ω is the frequency of relaxation.
- This collision model is called the *BGK model*.

Collision frequency and viscosity

- If ω is small, the fluid converges only slowly to its equilibrium: it is highly viscous.
- More generally, one can show that the fluid viscosity depends inversely on the relaxation parameter ω :

$$\nu = \delta t c_s^2 \left(\frac{1}{\omega} - \frac{1}{2} \right)$$

Collision: BGK model

The equilibrium is obtained as a truncated series of the Maxwell-Boltzmann distribution:

$$E(i, \rho, \mathbf{u}) = \rho t_i \left(1 + \frac{\mathbf{v}_i \cdot \mathbf{u}}{c_s^2} + \frac{1}{2 c_s^4} (\mathbf{v}_i \cdot \mathbf{u})^2 - \frac{1}{2 c_s^2} |\mathbf{u}|^2 \right)$$

- The constants t_i compensate for the different lengths of velocities $\mathbf{v}_i$.
- t_i is 1/9 for orthogonal directions, 1/36 for diagonal directions, and 4/9 for the rest velocity:

$$t = \text{array}([1/36, 1/9, 1/36, 1/9, 4/9, 1/9, 1/36, 1/9, 1/36])$$

Collision: Python codes

Equilibrium:

$$E(i, \rho, \mathbf{u}) = \rho t_i \left(1 + \frac{\mathbf{v}_i \cdot \mathbf{u}}{c_s^2} + \frac{1}{2 c_s^4} (\mathbf{v}_i \cdot \mathbf{u})^2 - \frac{1}{2 c_s^2} |\mathbf{u}|^2 \right)$$

We write the code in
«lattice units», i.e.
with $\delta x = \delta t = 1$.

```
def equilibrium(rho, u):  
    usqr = 3/2 * (u[0]**2 + u[1]**2)  
    eq = zeros((9, nx, ny))  
    for i in range(9):  
        vu = 3 * (v[i,0]*u[0,:,:] + v[i,1]*u[1,:,:])  
        eq[i,:,:] = rho*t[i] * (1 + vu + 0.5*vu**2 - usqr)  
    return eq
```


Collision: Python codes

Collision:

$$f_i^{\text{out}} - f_i^{\text{in}} = -\omega \left(f_i^{\text{in}} - E(i, \rho, \vec{u}) \right)$$

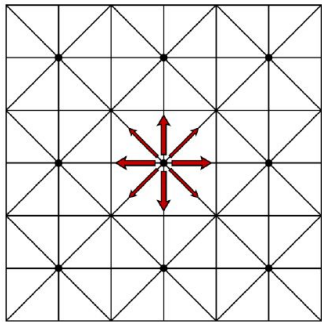
$$f_{out} = f_{in} - \omega * (f_{in} - eq)$$

7. Streaming Step

Streaming step

Streaming takes the system to the next time iteration, $t + \delta t$

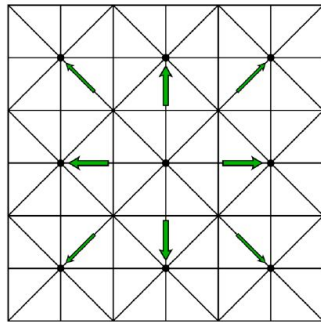
Time t



Post-collision f^{out}



Time $t + \delta t$



Pre-collision f^{in}



Streaming step: code

Conventional Python code:

- On boundaries, we apply periodicity.
- If a population leaves the domain, it enters the domain again on the other side.

```
for ix in range(nx):  
    for iy in range(ny):  
        for i in range(9):  
            next_x = ix + v[i,0]  
            if next_x < 0: next_x = nx-1  
            if next_x >= nx: next_x = 0  
  
            next_y = iy + v[i,1]  
            if next_y < 0: next_y = ny-1  
            if next_y >= ny: next_y = 0  
            fin[i, next_x,next_y] = fout[i, ix, iy]
```

Streaming step: code

Conventional Python code:

```
for ix in range(nx):
    for iy in range(ny):
        for i in range(9):
            next_x = ix + v[i,0]
            if next_x < 0: next_x = nx-1
            if next_x >= nx: next_x = 0

            next_y = iy + v[i,1]
            if next_y < 0: next_y = ny-1
            if next_y >= ny: next_y = 0
            fin[i, next_x, next_y] = fout[i, ix, iy]
```

NumPy Array-based code:

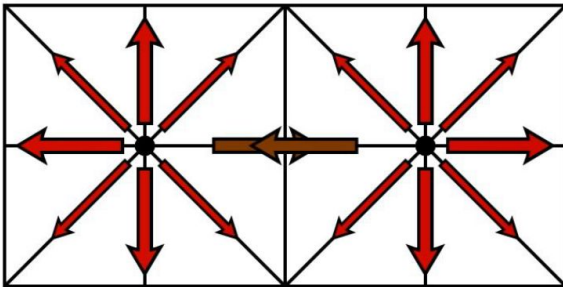
```
for i in range(9):
    fin[i,:,:] = roll(
        roll(fout[i,:,:], v[i,0], axis=0),
        v[i,1], axis=1 )
```

Do we really need double the memory?

- So far we allocated two sets of populations f^{in} and f^{out} .
- Could we spare one, and store both pre-collision and post-collision in the same variable f ?
- Answer: yes, but it needs some care.
- I will show how to do it. But for simplicity, we keep our code with two sets of populations.

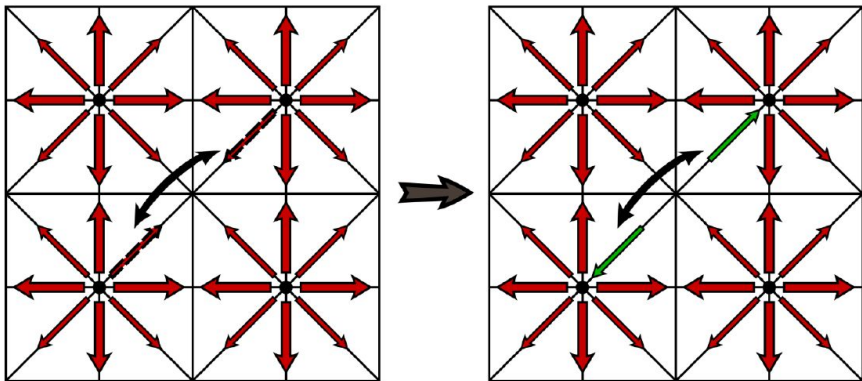
Single-population implementation

- Collision step, no problem: it is local. Simply overwrite f^{in} with f^{out} .
- Streaming step, more tricky. As you stream populations to a neighbor, you risk overwriting one that is still needed.
- Solution: *stream to the neighbor* and *from the neighbor* at the same time, exchanging populations instead of overwriting them.



Single-population implementation

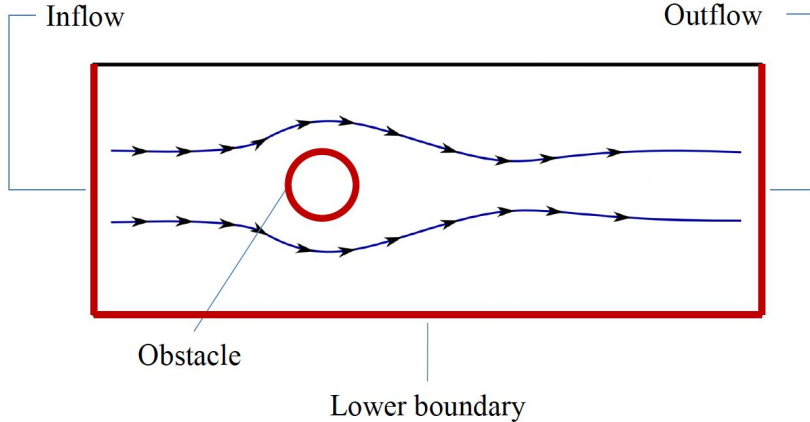
This strategy works along all directions:



8. Boundary conditions

Boundary conditions

Boundary conditions



Boundary conditions

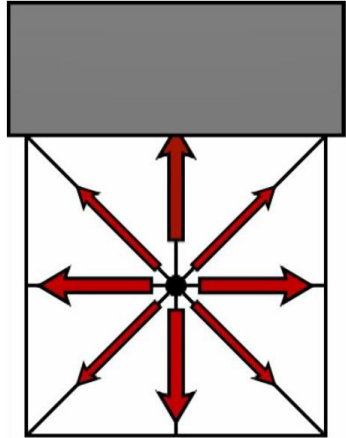
- We need different types of boundary conditions:
 - Inflow: imposes the velocity profile.
 - Outflow, and lower and upper boundary: pretend that the domain is larger.
 - Physical obstacle.
- There exists already a boundary condition in our code: periodicity.
 - Behaves as if our simulation was repeated an infinitely many times.
 - We will use periodicity for lower/upper boundary.

Obstacle: no-slip wall

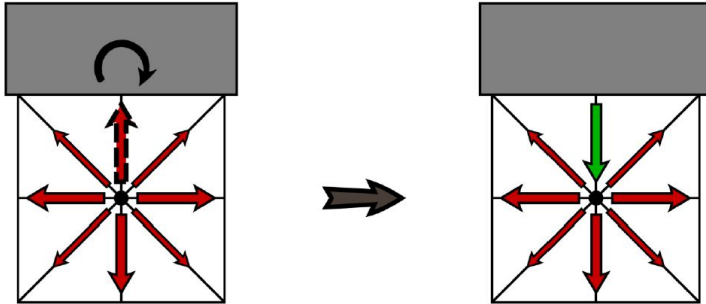
- Most physical obstacles act like no-slip walls.
- Macroscopically: the velocity is zero along the wall.
- At the molecular level: the obstacle has a surface roughness, and molecules «adhere» to the surface.
- Not all obstacles have a no-slip condition. Example: an air bubble in water is smooth, even at molecular level.

No-slip walls: bounce-back condition

- The bounce-back condition is a way to implement a no-slip wall in lattice Boltzmann.
- During streaming, a population which leaves the fluid and hits an obstacle «bounces back».
- Its value is unchanged, and it is copied to the population with reverse direction.

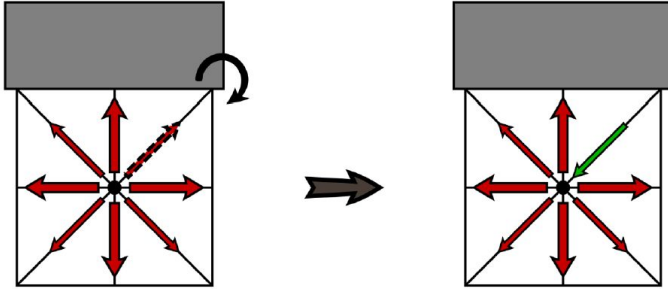


No-slip walls: bounce-back condition



This condition mimicks the physical process that occurs when molecules hit a wall.

No-slip walls: bounce-back condition



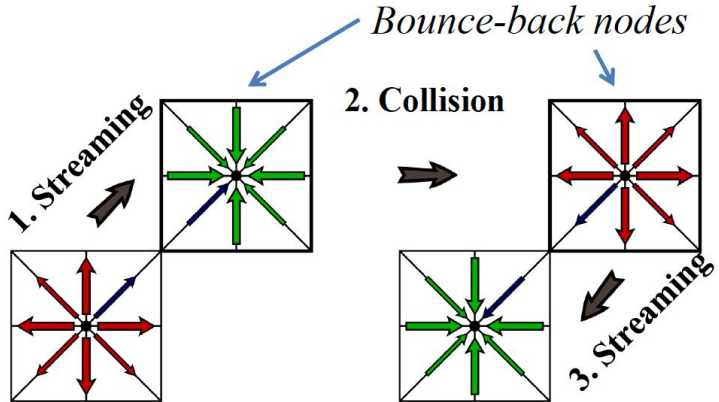
- Attention: populations on diagonal directions are not reflected like on a mirror. They also bounce-back to where they came from.
- This reflects the fact that at a molecular level, the obstacle is rough, and not smooth like a mirror.

No-slip walls: bounce-back condition

- Bounce-back is simple: track all populations that hit an obstacle, and send them back.
- Yet, this involves quite some book-keeping. Which nodes are affected? Which directions?
- Simpler solution: let the populations travel into the obstacle. Then, we revert them inside the obstacle, and send them back.
- No bookkeeping of directions. All cells inside an obstacle are bounce-back cells: instead of colliding, they revert all the directions.

The bounce-back cycle

During collision, a bounce-back node replaces every population by the one in the opposite direction.



The bounce-back cycle

Instead of colliding, bounce-back nodes implement the following relation:

$$f_i^{\text{in}}(\boldsymbol{x}, t + 1) = f_j^{\text{out}}(\boldsymbol{x}, t)$$

where direction i is opposite to direction j :

$$v_i = -v_j$$

Bounce-back:code

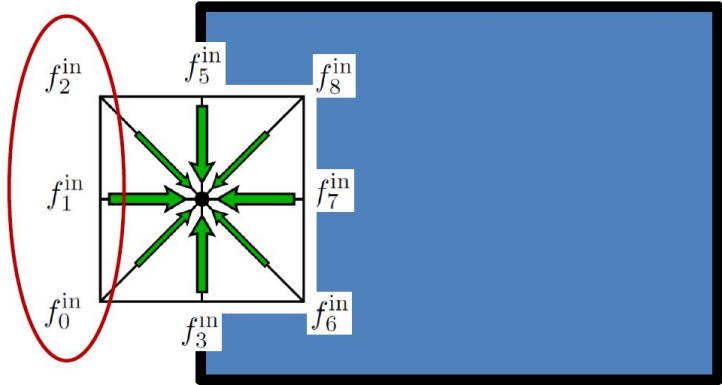
NumPy array-based code:

```
for i in range(9):  
    fout[i, obstacle] = fin[8-i, obstacle]
```

- We have on purpose listed the lattice velocities in such a way that the direction opposite to i is given by $8-i$.

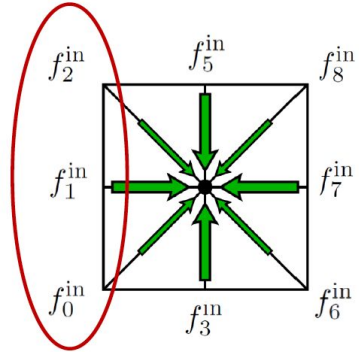
Inflow condition

After streaming,
the populations f_0 , f_1 , and f_2 are
unknown on all
cells along the left
boundary.



Inflow condition

- We want to impose a velocity $\mathbf{u}$ on inflow boundary cells. So, $\mathbf{u}$ is known. But what about ρ ? And what about f_0, f_1 , and f_2 ?
- We must extract this missing information using the known populations, and using $\mathbf{u}$.



Inflow condition: density

Remember:

$$\rho(\mathbf{x}, t) = \sum_{i=0}^8 f_i^{\text{in}}(\mathbf{x}, t)$$

$$\mathbf{u}(\mathbf{x}, t) = \frac{1}{\rho(\mathbf{x}, t)} \frac{\delta x}{\delta t} \sum_{i=0}^8 \mathbf{v}_i f_i^{\text{in}}(\mathbf{x}, t)$$

Let's define:

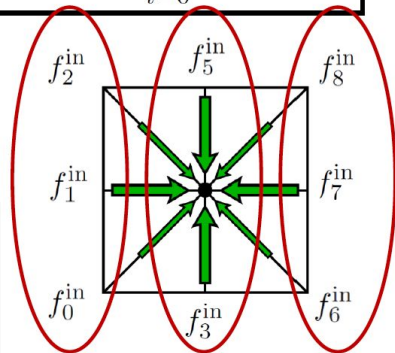
- $\rho_1 = f_0 + f_1 + f_2$ (unknown)
- $\rho_2 = f_3 + f_4 + f_5$ (known)
- $\rho_3 = f_6 + f_7 + f_8$ (known)

Then:

- $\rho = \rho_1 + \rho_2 + \rho_3$
- $\rho u_x = \rho_1 - \rho_3$

And hence:

$$\rho = \frac{\rho_2 + 2\rho_3}{1 - u_x}$$



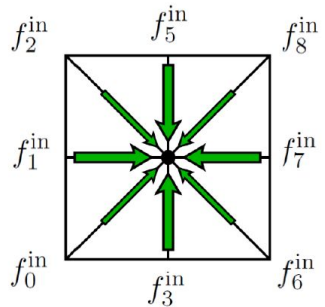
Inflow condition: unknown populations

- Remember: populations are always close to their equilibrium.
- We first initialize the unknown populations to their equilibrium value.
- Then, we check how much the opposite population deviates from equilibrium, at add this value as a correction.

$$f_0^{\text{in}} = E(0, \rho, \mathbf{u}) + (f_8^{\text{in}} - E(8, \rho, \mathbf{u}))$$

$$f_1^{\text{in}} = E(1, \rho, \mathbf{u}) + (f_7^{\text{in}} - E(7, \rho, \mathbf{u}))$$

$$f_2^{\text{in}} = E(2, \rho, \mathbf{u}) + (f_6^{\text{in}} - E(6, \rho, \mathbf{u}))$$



Inflow condition: code

- Define the indices of the three columns:
- Calculate the density:

```
col1 = array([0, 1, 2])  
col2 = array([3, 4, 5])  
col3 = array([6, 7, 8])
```

```
rho[0,:] = 1/(1-u[0,0,:]) * ( sum(fin[col2,0:], axis=0) +  
                               2*sum(fin[col3,0:], axis=0) )
```

- Calculate the populations:

```
fin[[0,1,2],0,:] = feq[[0,1,2],0,:] + fin[[8,7,6],0,:] - feq[[8,7,6],0,:]
```


Outflow condition

- Define the indices of the three columns:
- Calculate the density:

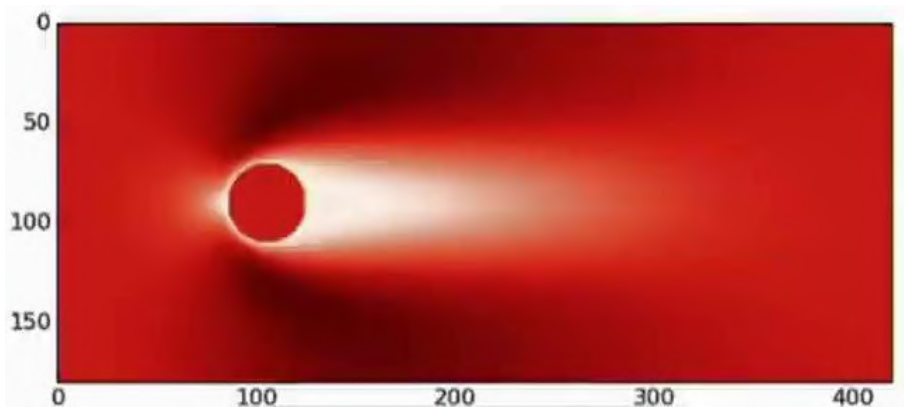
```
col1 = array([0, 1, 2])  
col2 = array([3, 4, 5])  
col3 = array([6, 7, 8])
```

```
rho[0,:] = 1/(1-u[0,0,:]) * ( sum(fin[col2,0:], axis=0) +  
                               2*sum(fin[col3,0:], axis=0) )
```

- Calculate the populations:

```
fin[[0,1,2],0,:] = feq[[0,1,2],0,:] + fin[[8,7,6],0,:] - feq[[8,7,6],0,:]
```

Results $Re=10$



Results $Re=220$

